

Day 4 — How Can Bits Become a Negative Number?

Curriculum alignment: Stage 1 — Foundations of Computation · Signed Numbers and Bitwise Reasoning

Target time: ~2 hours (≈50 minutes reading, ≈70 minutes experimenting)

A register contains 11111111. Is that 255 or -1?

The honest answer is: **show me the interpretation rule.**

Why negative representation became a machine-design problem

Human notation gets a separate minus symbol almost for free. A machine does not. Every extra symbol needs a physical state, a detector, and rules for how arithmetic machinery responds to it.

Early mechanical calculators made this cost visible. A wheel naturally cycles through a fixed set of positions. When it moves past its final digit, it wraps around and carries into the next wheel. Designers learned to exploit complement arithmetic because subtraction could then be transformed into addition plus discarded carry—much easier than building an entirely separate physical process for every signed case.

Binary hardware inherits the same circular structure:

```
decimal wheel: ... 8 → 9 → 0 → 1 ...  
8-bit register: ... 254 → 255 → 0 → 1 ...
```

Two's complement was not chosen because its bit patterns look intuitive. It won because it fits fixed-width arithmetic machinery:

- one representation for zero,
- addition and subtraction through the same adder,
- simple sign testing,
- and straightforward widening through sign extension.

This lesson is one step toward the CPU's arithmetic unit. Before we build an adder from gates, we need to understand the encoding that lets one adder serve both positive and negative calculations.

1. Begin with the problem, not two's complement

Days 1–3 established that bits have no inherent meaning. That idea now becomes practical.

An eight-bit storage location offers 256 patterns:

```
00000000 through 11111111
```

Unsigned binary assigns all 256 patterns to non-negative values:

```
00000000 → 0  
00000001 → 1  
...  
11111111 → 255
```

But programs need negative values. The hardware does not grow a minus-sign cell beside every register, so we need an encoding that divides the same 256 patterns between positive and negative numbers.

The question is not “How does the CPU store a minus sign?” It is:

Which mapping lets ordinary binary hardware perform signed arithmetic with the least extra machinery?

2. The first tempting idea: signed magnitude

Use the highest bit as a sign:

```
0xxxxxxx → positive  
1xxxxxxx → negative
```

For four bits:

```
0011 → +3  
1011 → -3
```

This resembles human notation. Unfortunately, it creates two zeros:

```
0000 → +0  
1000 → -0
```

Arithmetic also becomes awkward. Addition hardware must inspect signs and choose whether to add magnitudes or subtract them.

Signed magnitude is intuitive to read but inconvenient to compute with.

3. A second attempt: one's complement

Represent a negative value by flipping every bit of its positive counterpart:

```
+3 = 0011  
-3 = 1100
```

This improves some arithmetic behavior, but still has two zeros:

```
+0 = 0000
-0 = 1111
```

Addition also needs an “end-around carry” correction. Better, but not yet beautiful.

4. The representation that fits the hardware

Two’s complement forms $-x$ by:

1. invert every bit,
2. add one.

Using four bits:

```
+3      0011
invert   1100
add one  + 0001
         ----
-3      1101
```

Why the extra one? Because it removes negative zero and makes the patterns form one continuous modular number circle.

Four bits give 16 positions:

```
0000  0
0001  1
...
0111  7
1000 -8
1001 -7
...
1111 -1
```

There is exactly one zero, and every pattern has a job.

5. Stop thinking “special negative bits”

Two’s complement becomes easier when viewed as arithmetic modulo 2^n .

With four bits, calculations retain only the lowest four bits. Values wrap around a circle of size 16:

```
          0000 (0)
        1111 (-1)  0001 (1)
      1110 (-2)    0010 (2)
    1101 (-3)      0011 (3)
    1100 (-4)      0100 (4)
      ...          ...
          1000 (-8)
```

To encode -3 , calculate:

$$2^4 - 3 = 16 - 3 = 13 = 1101_2$$

The pattern **1101** can therefore be read as:

- unsigned: 13
- four-bit two's complement: -3

The bits did not change. The mathematical interpretation did.

6. Why the same adder works

Try **5 + (-3)** in four bits:

```
+5  0101
-3  1101
----
    10010
```

A four-bit register keeps the low four bits:

0010 → 2

The carry leaving the register is discarded. Ordinary binary addition produced the correct signed result without a separate “negative-number adder.”

That is the engineering victory of two's complement:

Addition and subtraction work through the same fixed-width binary adder, regardless of whether software later calls the operands signed or unsigned.

The CPU still needs status flags to tell us whether a result was valid under a particular interpretation. We will meet those on Day 6.

7. Reading a two's-complement value

For an **n**-bit value:

- if the highest bit is 0, read it normally;
- if the highest bit is 1, its place value is -2^{n-1} .

For eight bits:

bit:	7	6	5	4	3	2	1	0
place value:	-128	64	32	16	8	4	2	1

Read **11110110**:

$$-128 + 64 + 32 + 16 + 0 + 4 + 2 + 0 = -10$$

Or invert and add one:

```
11110110
invert → 00001001
+1      → 00001010 = 10
therefore original value = -10
```

Both methods describe the same encoding.

8. The asymmetric range

Eight-bit unsigned range:

```
0 through 255
```

Eight-bit signed two's-complement range:

```
-128 through +127
```

Why one extra negative value?

There are 256 patterns and only one zero. The patterns divide into:

```
128 negative + 1 zero + 127 positive = 256
```

The most negative value, `10000000`, has no positive counterpart representable in the same width. This asymmetry later creates a famous edge case:

```
abs(INT_MIN)
```

Its positive mathematical result does not fit in the same signed type.

9. Observe the same byte two ways in Python

Python's `int.from_bytes` lets us choose the interpretation explicitly:

```
python3 - <<'PY'
raw = bytes([0b11110110])
print("stored byte:", raw.hex())
print("unsigned:", int.from_bytes(raw, byteorder="little", signed=False))
print("signed: ", int.from_bytes(raw, byteorder="little", signed=True))
PY
```

Expected:

```
stored byte: f6
unsigned: 246
signed:   -10
```

This is Day 2's lesson becoming executable: identical storage, different interpretation argument.

Now replace `0b11110110` with:

```
01111111
10000000
11111111
```

Predict both readings before running.

10. A C observation

Create `signed_view.c`:

```
#include <stdint.h>
#include <stdio.h>

int main(void)
{
    uint8_t bits = 0xf6;
    int8_t signed_view = (int8_t)bits;

    printf("bits as hex:      0x%02x\n", bits);
    printf("unsigned view:    %u\n", bits);
    printf("signed view:      %d\n", signed_view);
    return 0;
}
```

Build and run:

```
gcc -std=c17 -Wall -Wextra -O0 -g signed_view.c -o signed_view
./signed_view
```

Then inspect:

```
objdump -d -Intel signed_view
```

Do not expect RAM to contain separate unsigned and signed copies. The program changes how the compiler extends and formats the value. One language detail: converting an out-of-range unsigned value to a signed type is implementation-defined in C17. GCC on this x86-64 system produces the two's-complement reading shown here; do not silently generalize that cast to every C implementation.

11. Try the interactive signed-byte view

The Day 2 Bit Lab already contains a signed and unsigned view of one byte:

```
~/masterclass/daily/challenges/day-002-bit-lab.html
```

Toggle bit 7 while keeping the lower seven bits fixed.

Ask:

- Why does unsigned jump upward?
- Why does signed jump from positive to negative?
- Which interpretation is “really inside” the byte?

The answer to the last question is: neither. The byte contains the pattern.

12. Checkpoint

Explain without using the phrase “that’s just how negatives are stored”:

1. Why signed magnitude creates two zeros.
2. How invert-plus-one forms a negative two’s-complement value.
3. Why ordinary fixed-width addition works for both signed and unsigned interpretations.
4. Why `11111111` can mean 255 or -1 .
5. Why an eight-bit signed range is -128 through $+127$.

Prediction challenge

Without a calculator:

Eight-bit pattern: `11100101`

Write:

- its unsigned value,
- its signed two’s-complement value,
- the steps used for each.

Do not move on based on recognizing the term “two’s complement.” Move on when you can derive the mapping.

References used selectively

- Patt and Patel, *Introduction to Computing Systems*, 2nd ed., Chapter 2, especially “Integer Data Types,” conversion, arithmetic, sign extension, and overflow (book pages 21–33).
- Bryant and O’Hallaron, *Computer Systems: A Programmer’s Perspective*, 3rd ed., Chapter 2, §2.2 “Integer Representations.”
- Charles Petzold, *Code*, Chapter 13, “But What About Subtraction?”
- Computer History Museum, “Computers Timeline,” for the progression from mechanical arithmetic to relay calculators: <https://www.computerhistory.org/timeline/computers/>

Next: Day 5 turns individual bits into controls: masks, AND, OR, XOR, NOT, shifts, and the register-field patterns used throughout kernel and driver code.